PMSCS 622P
Programming Languages

Lecture 01: Introduction to Computer Programming

Mr. Md. Masum Bhuiyan

Department of Computer Science and Engineering
Jahangirnagar University

1.1 What is Computer Programming?

Definition

Computer programming is the act of designing and building an executable computer program to accomplish a specific computing result.

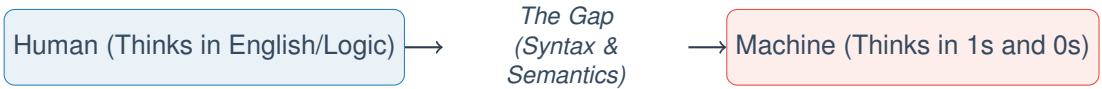
It is a two-step process:

1. **Problem Solving:** Creating the logic (Algorithm) to solve a problem.
2. **Coding:** Translating that logic into a language the machine understands.

"Computers are incredibly fast, accurate, and stupid. Humans are incredibly slow, inaccurate, and brilliant. Together they are powerful." - Albert Einstein

The Communication Gap

Why do we need programming languages?



- **Natural Language:** Ambiguous ("I saw the man with the telescope").
- **Machine Language:** Exact (01001000).
- **Programming Language:** A formal compromise (Strict rules, readable by humans).

1.2 What is Source Code?

Source Code is the text written by a programmer in a High-Level Language (like C++, Java, Python).

- It is human-readable.
- It follows specific grammar rules called **Syntax**.
- It cannot be executed directly by the hardware.

Example (Python)

```
print("Hello, World!")
```

Machine Code

Machine Code is the lowest level of software instructions.

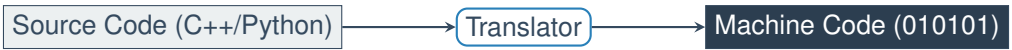
- It consists entirely of binary digits (0 and 1).
- It controls the CPU (Central Processing Unit) directly.
- It is specific to the hardware (Intel machine code looks different from ARM machine code).

Example (Binary for "Hello")

```
01001000 01100101 01101100 01101100 01101111
```

The Translator

A **Translator** is a system software that converts High-Level Source Code into Low-Level Machine Code.



There are two main types of translators:

1. **Compiler**
2. **Interpreter**

The Compiler (e.g., C++)

A compiler translates the **entire** program at once into a separate file (Executable).

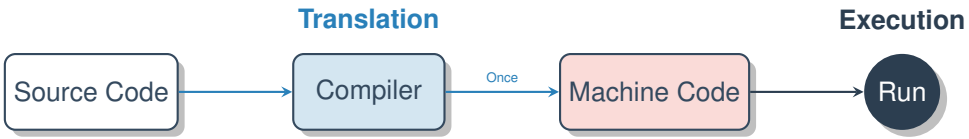
- **Process:** Read all code $\rightarrow$ Check for errors $\rightarrow$ Build `.exe` file.
- **Analogy:** A translator translating a whole English book into Bengali before anyone reads it.
- **Pros:** Execution is very fast (CPU runs the binary directly).
- **Cons:** If you change one line, you must re-compile the whole program.

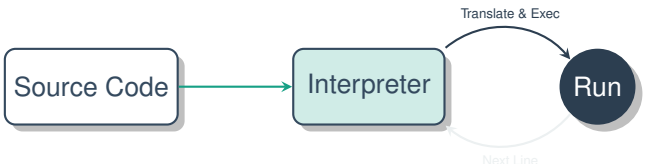
The Interpreter (e.g., Python)

An interpreter translates and executes the code **line-by-line**.

- **Process:** Read Line 1 $\rightarrow$ Execute $\rightarrow$ Read Line 2 $\rightarrow$ Execute.
- **Analogy:** A simultaneous interpreter at the UN translating a speech live as it is spoken.
- **Pros:** Easy to test and debug (stops exactly where the error is).
- **Cons:** Slower execution (translation happens every time you run it).

Visual Comparison: Execution Flow





1.3 The Compilation Process

When you click "Build" in C++, several invisible steps occur to turn your text into software.

The three main stages are:

1. **Preprocessing:** Handling "starts with #" commands.
2. **Compiling:** Converting syntax to machine instructions (Object code).
3. **Linking:** Combining all parts into the final application.

Step 1-2: Preprocessing & Compiling

1. Preprocessing (The Cleaner)

Before compiling, the computer follows commands starting with `#`.

- Example: `#include <iostream>` copies the contents of the header file into your program.
- It cleans up the code for the compiler.

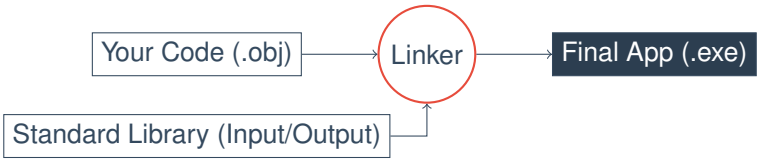
2. Compiling (The Translator)

The compiler checks for **Syntax Errors** (spelling mistakes in code).

- If error free: It generates an **Object File** (`.obj` or `.o`).
- This is machine code, but it is incomplete (missing library parts).

Step 3: Linking

The Linker connects your code with external libraries.



If you use `cout` (print), the code for how to print is not in your file. The Linker grabs it from the OS libraries and attaches it to your program.

Types of Errors

During these processes, things can go wrong.

Error Type	Definition	Example
Syntax Error	Violation of grammar rules. Caught by Compiler.	Missing a semicolon (;).
Logical Error	The program runs, but gives the wrong result. Caught by Human.	Calculating $Area = 2 + r$ instead of πr^2.
Runtime Error	Program crashes while running.	Dividing by zero.

1.4 What is an Algorithm?

Definition

An **Algorithm** is a step-by-step procedure or set of rules to solve a specific problem.

It is **Language Independent**. You can write an algorithm in English, and then code it in C++, Java, or Python.

Real World Example (Tea Algorithm):

1. Boil water.
2. Add tea leaves.
3. Pour into cup.
4. Add sugar.

Characteristics of a Good Algorithm

1. **Input:** It may accept zero or more inputs.
2. **Output:** It must produce at least one result.
3. **Definiteness:** Each step must be clear and unambiguous.
4. **Finiteness:** The algorithm must stop after a finite number of steps (no infinite loops).
5. **Effectiveness:** Each step must be simple enough to be done.

Pseudocode

Pseudocode (False Code) is a way of describing an algorithm using a mix of plain English and code-like structure.

- It ignores strict syntax (no semicolons needed).
- It helps programmers plan logic before worrying about grammar.

Example: Add Two Numbers

```
START
READ number1, number2
sum = number1 + number2
PRINT sum
END
```

Control Flow Graph (CFG)

A **Control Flow Graph (CFG)** is a representation of all paths that might be traversed through a program during its execution.

- **Nodes:** Represent basic blocks (a sequence of non-jump instructions).
- **Edges:** Represent control flow paths (jumps, branches).

The Rules:

1. Assign a **Node Number** to each expression.
2. Draw a directed edge $A \rightarrow B$ if B executes after A.
3. For branches, mark edges as **True** or **False**.

Practical Example

Problem: Write a program to check if a student Passed or Failed. (Pass Mark = 40).

We will look at:

1. The Algorithm.
2. The Pseudocode.
3. The Control Flow Graph.

Solution: Logic

Algorithm (English)

1. Start.
2. Ask student for marks.
3. Check if marks are 40 or more.
4. If yes, say "Pass".
5. If no, say "Fail".
6. Stop.

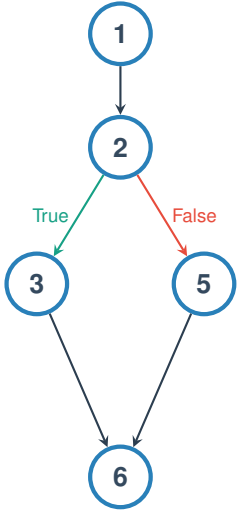
Pseudocode

```
START
INPUT marks
IF marks >= 40 THEN
  PRINT "Pass"
ELSE
  PRINT "Fail"
ENDIF
END
```

Process of Creating a CFG

Sample Code:

```
(1) input x
(2) if x > 10:
(3)   print "High"
(4) else:
(5)   print "Low"
(6) print "End"
```



2.3 Complexity Analysis: Time & Space

Why Analyze Algorithms?

To predict how an algorithm performs as the input size (n) grows to infinity.

1. Time Complexity

- **Not** measured in seconds (too hardware dependent).
- Measured in the **number of operations** executed.
- Example: Comparing n vs n^2 steps.

2. Space Complexity

- Amount of memory (RAM) required.
- Includes inputs + auxiliary variables.
- Example: Storing an array of size n requires $O(n)$ space.

1. Big-O Notation (O)

Definition (Exam Standard)

Big-O notation (O) describes the **Worst-Case Scenario**. It defines the maximum time an algorithm will take.

$$f(n) = O(g(n))$$

Illustrative Example: Linear Search

- **Problem:** Find value 7 in list $[1, 2, 3, 4, 5, 6, 7]$.
- **Scenario:** The number is at the very end.
- **Work:** We must check all n elements.
- **Result:** $O(n)$ (Linear Time).

2. Big-Omega Notation (Ω)

Definition

Big-Omega (Ω) describes the **Best-Case Scenario**. It defines the minimum time required.

Illustrative Example: Linear Search

- **Problem:** Find value 1 in list $[1, 2, 3, 4, 5]$.
- **Scenario:** The number is at the very first index.
- **Work:** We check only 1 element.
- **Result:** $\Omega(1)$ (Constant Time).

Note: In exams, unless specified, we usually care about Worst Case (O).

3. Big-Theta Notation (Θ)

Definition

Big-Theta (Θ) describes the **Average/Tight Case**. It bounds the function from both above and below.

When to use?

- When the Best Case and Worst Case are strictly different (like Linear Search), Θ is complex.
- When the algorithm always takes the same time (like accessing an array index), $O(1) = \Omega(1) = \Theta(1)$.

How to Calculate Complexity (Code Example)

Question: Calculate the Time Complexity of the following snippet.

```
void f(int n) {
    int a = 0;              // Cost: 1
    for (int i=0; i<n; i++) {
        a = a + 1;          // Cost: n
    }
    for (int j=0; j<n; j++) {
        for (int k=0; k<n; k++) {
            cout << k;      // Cost: n * n
        }
    }
}
```

Analysis:

$$Total = 1 + n + n^2$$

Rules for Big-O:

1. Drop constants ($1 \to 0$).
2. Drop lower order terms (n is smaller than n^2).
3. Keep the dominant term.

Answer: $O(n^2)$ **(Quadratic)**

Lecture Summary Key Takeaways

1. **The Core Bridge:** Programming translates Human Logic (Algorithms) into Machine Reality (Binary).
2. **Translation Models:**
   - **Compiler:** Optimized for speed (Batch translation).
   - **Interpreter:** Optimized for flexibility (Line-by-line).
3. **The "Invisible" Build:** `Preprocessing` $\rightarrow$ `Compiling` $\rightarrow$ `Linking` (Connects Libraries).
4. **Control Flow:**
   - **Flowcharts** help design logic.
   - **CFGs** map execution paths (Nodes = Code, Edges = Jumps).
5. **Analysis:** We use **Big-O (O)** to predict Worst-Case performance ($Time$ vs $Space$).